Detailed Project Plan (4 Weeks)

Project Title: Medical Clinic Digital Twin (BLE-Based Patient Tracking)

---

1. Brief Restatement of Project Idea (Max One Page)

The **Medical Clinic Digital Twin** is an IoT-based smart healthcare monitoring system designed to improve patient flow management inside clinics and hospitals. In many healthcare facilities, patients spend excessive time waiting between reception, consultation rooms, diagnostic labs, and pharmacies. These delays often occur because administrators lack real-time visibility into patient movement and occupancy levels.

Our project solves this problem by creating a **real-time digital twin** of the clinic environment using **Bluetooth Low Energy (BLE) patient tags** and **ESP32 gateway devices**. Each patient receives a BLE tag upon arrival. Multiple ESP32 gateways installed in strategic clinic zones continuously scan for BLE advertisements and detect patient proximity using RSSI values.

The collected data is transmitted securely to a cloud server using MQTT over TLS, where it is processed to determine patient location, movement history, waiting durations, congestion points, and abnormal delays. A web dashboard visualizes the clinic layout and displays live occupancy statistics, waiting times, bottleneck alerts, and analytics.

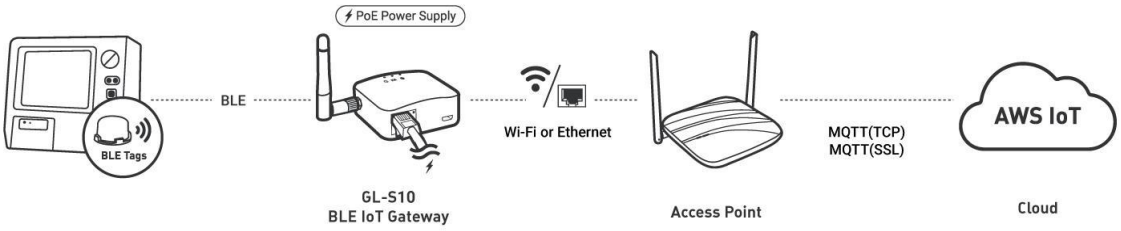
The system aims to provide the following benefits:

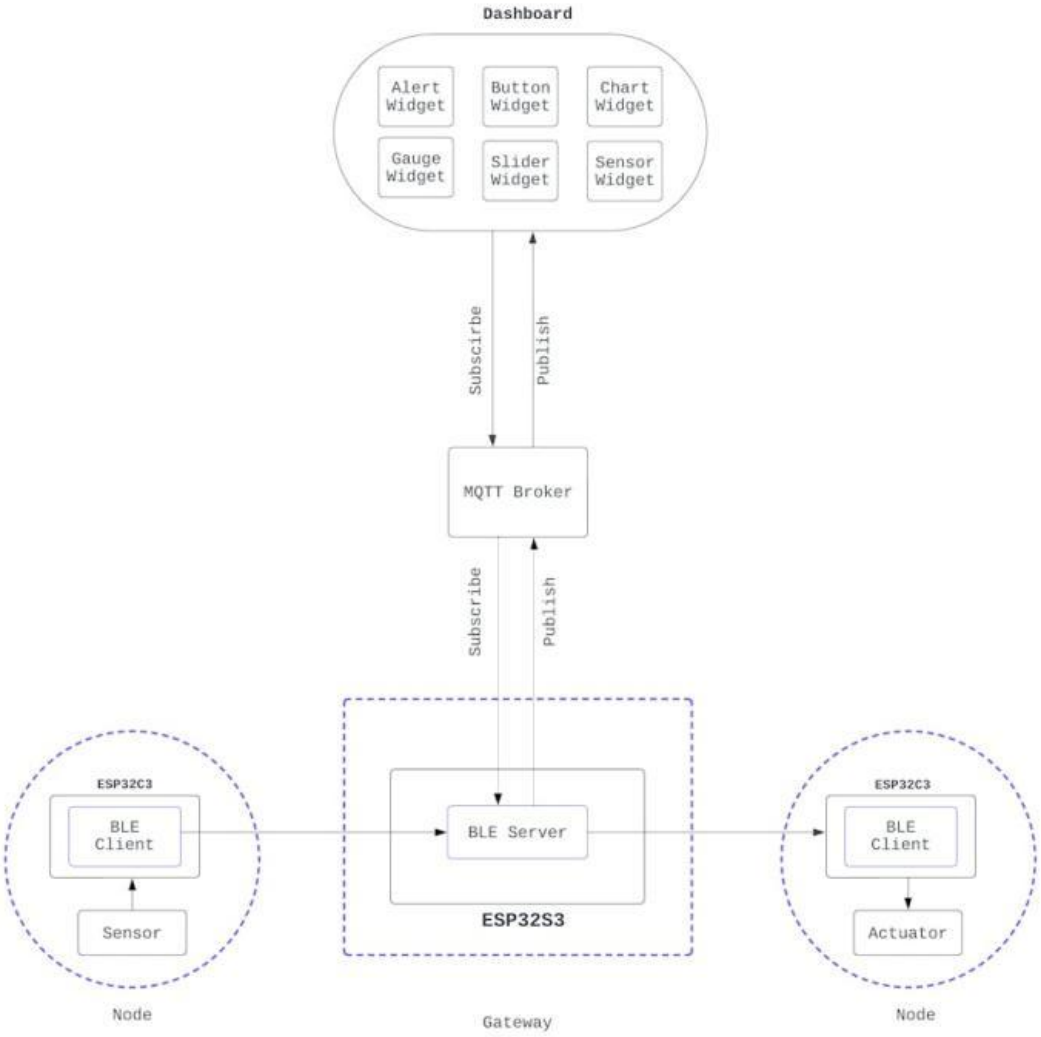
- Reduce patient waiting times
- Improve staff allocation and scheduling
- Detect overcrowded areas early
- Increase clinic operational efficiency
- Enable data-driven management decisions
- Demonstrate practical Digital Twin technology in healthcare

This project combines embedded systems, IoT networking, cloud computing, web development, and machine learning into one integrated smart healthcare solution.

---

2. Full System Architecture (Max One Page)


PoE Power Supply
BLE
BLE Tags
GL-S10
BLE IoT Gateway
Wi-Fi or Ethernet
Access Point
MQTT(TCP)
MQTT(SSL)
AWS IoT
Cloud


Dashboard
Alert Widget
Button Widget
Chart Widget
Gauge Widget
Slider Widget
Sensor Widget
Subscirbe
Publish
MQTT Broker
Subscribe
Publish
ESP32C3
BLE Client
Sensor
Node
BLE Server
ESP32S3
Gateway
ESP32C3
BLE Client
Actuator
Node

Architecture Layers

A. Perception Layer (End Devices)

- BLE wearable tags assigned to patients
- Broadcast unique patient IDs periodically

B. Edge Layer

- ESP32 gateway nodes installed in:
    - Reception
    - Waiting Area
    - Examination Rooms
    - Pharmacy
- Scan BLE packets and calculate RSSI

C. Network Layer

- Wi-Fi connectivity from ESP32 nodes to cloud server
- MQTT over TLS for secure lightweight communication

D. Processing Layer

Cloud backend responsible for:

- Receiving telemetry data
- Time synchronization
- Database storage
- Location estimation
- Analytics engine
- ML anomaly detection

E. Application Layer

Web dashboard displaying:

- Real-time patient locations

- Heatmaps of congestion zones
- Waiting time reports
- Daily/weekly statistics
- Alerts for delays or anomalies

Main Data Flow

BLE Tag → ESP32 Gateway → MQTT Broker → Backend API → Database → Dashboard

---

3. Four-Week Detailed Execution Plan

Start Date: Thursday, April 23

Week 1 (Apr 23 – Apr 29)

Objective: Hardware Setup + Core Communication Infrastructure

Tasks

Hardware Preparation

- Prepare ESP32 boards / Personal Laptops will be used instead.
- Prepare BLE tags / mobile BLE simulators
- Assign IDs to patient tags

Firmware Development

- Program ESP32 as BLE scanners
- Scan nearby BLE advertisements
- Capture:
    - Tag ID
    - RSSI
    - Timestamp
    - Gateway ID

Network Setup

- Configure clinic Wi-Fi test network
- Connect ESP32 nodes to internet

Cloud Setup

- Deploy MQTT broker (Mosquitto / HiveMQ)

Security Setup (Recommended)

- Enable TLS certificates
- Secure credentials

Documentation

- Hardware topology
- Gateway placement map

Deliverables End of Week 1

- Functional ESP32 gateways scanning BLE devices
- Successful MQTT data transmission to cloud
- Secured broker connection
- Initial firmware source code
- Hardware deployment report

Week 2 (Apr 30 – May 6)

Objective: Backend Development + Database + Location Engine

Tasks

Backend API

- Develop NodeJS backend
- Subscribe to MQTT topics
- Receive live gateway data

Database Design

- Tables:

    - routers
    - devices
    - records

Localization Engine (Recommended)

- Compare RSSI from multiple gateways
- Assign patient current zone
- Resolve duplicate detections

Session Tracking

- Entry / exit timestamps
- Time spent in each area

Testing (Recommended)

- Simulate 5–10 patients movement

Deliverables End of Week 2

- Live backend receiving data
- Working SQL database
- Real-time patient zone detection
- Patient movement history logs
- Localization algorithm v1

Week 3 (May 7 – May 13)

Objective: Dashboard + Analytics + Visualization

Tasks

Frontend Dashboard

Build responsive web dashboard with:

- Live clinic map
- Patient icons moving by zone

- Occupancy counters
- Average wait times

Reports Section

- Daily report
- Peak hours
- Queue durations
- Department efficiency

Visualization Features

- Heatmaps
- Charts
- Historical timelines

Real-Time Updates (Recommended)

- WebSocket integration

UI Testing

Deliverables End of Week 3

- Fully functional dashboard
- Live patient visualization
- Wait-time analytics reports
- Interactive charts
- UI demo ready

Week 4 (May 14 – May 20)

Objective: AI Features + Final Integration + Testing + Presentation

Tasks

Machine Learning Module (Recommended)

Train anomaly detection model for:

- Long waiting patients
- Unexpected movement loops
- Crowded zones
- Missing tags

System Integration

- Hardware + backend + frontend full sync

Stress Testing (Recommended)

- 20+ simultaneous tags
- Packet loss scenarios
- Network interruption recovery

Optimization (Recommended)

- Reduce latency
- Improve battery efficiency
- Tune RSSI thresholds

Final Documentation

Prepare:

- Final report
- Architecture diagrams
- User manual
- Source code documentation
- Slides

Final Demo Scenario

Complete simulated clinic workflow

Deliverables End of Week 4

- Fully integrated final system
- ML anomaly detection module

- Final dashboard polished version
- Stress test report
- Final report + presentation slides
- Ready-for-grading demo system

---

4. Expected Final Project Scope

This project includes:

- Embedded systems programming
- BLE wireless communication
- ESP32 networking
- Secure MQTT cloud communication
- Backend development
- SQL database engineering
- Real-time dashboard frontend
- Data analytics
- Machine learning anomaly detection
- System integration & testing

---

5. Team Work Distribution (Recommended)

- **Member 1:** ESP32 firmware + BLE scanning
- **Member 2:** MQTT + Cloud Backend
- **Member 3:** Database + APIs
- **Member 4:** Dashboard Frontend
- **Member 5:** ML + Testing + Documentation